

1. Greedy Knapsack

(a)

Algorithm: Sort items by weight ascending. Pick the lightest items one by one until the next item would exceed capacity.

```

begin
  UNITVALUEKNAPSACK( $W[1..n]$ ,  $C$ )
  Sort  $W$  in nondecreasing order
   $k \leftarrow 0$ ;  $w \leftarrow 0$ 
  while  $k < n$  and  $w + W[k + 1] \leq C$  do
     $k \leftarrow k + 1$ 
     $w \leftarrow w + W[k]$ 
  return  $k$ 

```

Proof (by contradiction): Let greedy select items $g_1, \dots, g_k$ with $W[g_1] \leq \dots \leq W[g_k]$ (the k lightest). Let OPT select m items; sort them by weight as $W[o_1] \leq \dots \leq W[o_m]$.

Since greedy chose the k lightest items overall, the j -th lightest item overall is at most the j -th lightest in any subset:

$$\forall j \leq m : W[g_j] \leq W[o_j]$$

Suppose $m > k$. Then:

$$\sum_{j=1}^m W[o_j] \geq \sum_{j=1}^{k+1} W[o_j] \geq \sum_{j=1}^{k+1} W[g_j] > C \quad (\text{greedy stopped at } k)$$

This contradicts OPT being feasible.

$$k \geq m \implies \text{greedy is optimal}$$

Runtime: Sort $O(n \log n)$ + scan $O(n) = O(n \log n)$

(b)

Counterexample: $n = 3$, $C = 2$, $W = [1, 1, 2]$, $V = [1, 1, 3]$.

i	1	2	3
$W[i]$	1	1	2
$V[i]$	1	1	3

Greedy: Sorted by weight: items 1, 2, 3. Pick items 1, 2 (weight $1 + 1 = 2 \leq C$). Value = 2.

OPT: Item 3 alone: weight $2 \leq C$, value = 3.

$$\text{Greedy} = 2 < 3 = \text{OPT}$$

(c)

Counterexample: $n = 2$, $C = 2$, $W = [1, 2]$, $V = [2, 3]$.

Greedy: Sorted by weight: items 1, 2. Pick item 1 (weight 1). Item 2 does not fit ($1 + 2 = 3 > 2$). Value = 2.

OPT: Item 2 alone: weight $2 \leq C$, value = 3.

$$\boxed{\text{Greedy} = 2 < 3 = \text{OPT}}$$

2. Modified Interval Scheduling

(a)

Counterexample: $I_1 = [0, 2)$ with $V_1 = 1$, $I_2 = [1, 3)$ with $V_2 = 2$, $I_3 = [3, 5)$ with $V_3 = 1$.

EFT greedy: Sort by finish time: I_1, I_2, I_3 . Pick I_1 (finish 2). I_2 conflicts ($S_2 = 1 < 2 = F_1$), skip. Pick I_3 ($S_3 = 3 \geq 2$). Value = $1 + 1 = 2$.

OPT: $\{I_2, I_3\}$: compatible ($F_2 = 3 \leq 3 = S_3$). Value = $2 + 1 = 3$.

$$\boxed{\text{Greedy} = 2 < 3 = \text{OPT}}$$

(b)

Greedy fails: $I_1 = [0, 1)$ with $V_1 = 1$, $I_2 = [0.5, 1.5)$ with $V_2 = 100$.

EFT picks I_1 (finish 1), then I_2 conflicts ($S_2 = 0.5 < 1$). Value = 1. OPT: $\{I_2\}$, value = 100.

Adapted algorithm: Since all intervals have unit length, $F[i] = S[i] + 1$, so sorting by start time is the same as sorting by finish time. We adapt the standard weighted interval scheduling DP:

```

begin
  UNITLENGTHSCHEDULING( $S[1..n]$ ,  $V[1..n]$ )
  Sort intervals by  $S[i]$ 
   $DP[0] \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
     $p[i] \leftarrow$  largest  $j \geq 0$  with  $S[j] + 1 \leq S[i]$            // binary search
     $DP[i] \leftarrow \max(DP[i-1], V[i] + DP[p[i]])$ 
  return  $DP[n]$ 

```

Correctness: $p[i]$ finds the latest non-conflicting interval ($F[j] = S[j] + 1 \leq S[i]$). $DP[i] = \max$ value using intervals $\{1, \dots, i\}$: either skip i ($DP[i-1]$) or take i ($V[i] + DP[p[i]]$).

Runtime: Sort $O(n \log n)$ + n binary searches $O(\log n)$ + DP fill $O(n)$ = $\boxed{O(n \log n)}$

3. Interval Coloring

Data structure: Sorted event array + stack of available room numbers.

```

begin
  INTERVALCOLORING( $S[1..n]$ ,  $F[1..n]$ )
  Create  $2n$  events: ( $S[i]$ , start,  $i$ ) and ( $F[i]$ , end,  $i$ ) for each  $i$ 
  Sort events by time (end before start at equal times)
   $R \leftarrow 0$ ; free  $\leftarrow$  empty stack
  for each event in sorted order do
    if event is end for interval  $j$  then
      free.PUSH(color[ $j$ ])
    else
      // start for interval  $j$ 
      if free  $\neq \emptyset$  then
        color[ $j$ ]  $\leftarrow$  free.POP() // reuse room
      else
         $R \leftarrow R + 1$ 
        color[ $j$ ]  $\leftarrow R$  // new room
  return  $R$ 

```

Correctness: Whenever a new room R is opened at time $S[i]$, the stack is empty, so all $R - 1$ existing rooms are in use. Each of those rooms holds an interval that started before $S[i]$ and has not yet ended, so it overlaps $S[i]$. Together with I_i itself, at least R intervals are active at time $S[i]$:

$$\text{depth}(S[i]) \geq R$$

So $R \leq D = \max_t \text{depth}(t)$: we never open more than D rooms. Conversely, any valid coloring needs $\geq D$ rooms, so $R \geq D$. Hence $R = D$. $\square$

Runtime: Create events $O(n)$ + sort $O(n \log n)$ + sweep $O(n)$ stack ops = $O(n \log n)$

4. Interval Coloring with Small and Large Rooms

Lower bound:

$$D = \max_t d(t) \quad (\text{max depth of all intervals})$$

$$D_1 = \max_t d_1(t) \quad (\text{max depth of } B[i] = 1 \text{ intervals})$$

$$R_S + R_L \geq D \quad (\text{total rooms} \geq \text{max depth})$$

$$R_L \geq D_1 \quad (\text{large rooms} \geq \text{max depth of large-only})$$

$$R_S + 2R_L = (R_S + R_L) + R_L \geq D + D_1$$

Algorithm: Two-pass event sweep. Pass 1 computes D . Pass 2 assigns rooms using two stacks free_L , free_S and array $\text{tag}[\cdot]$ tracking room type.

```

begin
  SMALLLARGECOLORING( $S[1..n]$ ,  $F[1..n]$ ,  $B[1..n]$ )
  Create  $2n$  events: ( $S[i]$ , start,  $i$ ) and ( $F[i]$ , end,  $i$ ) for each  $i$ 
  Sort events by time (end before start at equal times)

  // Pass 1: compute  $D$ 
   $D \leftarrow 0$ ; active  $\leftarrow 0$ 
  for each event in sorted order do
    if end then active  $\leftarrow$  active  $- 1$ 
    else active  $\leftarrow$  active  $+ 1$ ;  $D \leftarrow \max(D, \text{active})$ 

  // Pass 2: assign rooms
   $R_S \leftarrow 0$ ;  $R_L \leftarrow 0$ 
   $\text{free}_S, \text{free}_L \leftarrow$  empty stacks
  for each event in sorted order do
    if event is end for interval  $j$  then
      Push  $\text{color}[j]$  onto  $\text{free}_L$  or  $\text{free}_S$  based on  $\text{tag}[\text{color}[j]]$ 
    else if  $B[j] = 1$  // start, needs large room then
      if  $\text{free}_L \neq \emptyset$  then
         $\text{color}[j] \leftarrow \text{free}_L.\text{POP}()$  // reuse large
      else if  $\text{free}_S \neq \emptyset$  then
         $r \leftarrow \text{free}_S.\text{POP}()$ 
         $\text{tag}[r] \leftarrow L$  // promote small  $\rightarrow$  large
         $R_S \leftarrow R_S - 1$ ;  $R_L \leftarrow R_L + 1$ 
         $\text{color}[j] \leftarrow r$ 
      else
         $R_L \leftarrow R_L + 1$ 
         $\text{color}[j] \leftarrow R_S + R_L$ ;  $\text{tag}[\text{color}[j]] \leftarrow L$  // new large
    else // start,  $B[j] = 0$ , needs any room
      if  $\text{free}_S \neq \emptyset$  then
         $\text{color}[j] \leftarrow \text{free}_S.\text{POP}()$  // reuse small
      else if  $R_S + R_L < D$  then
         $R_S \leftarrow R_S + 1$ 
         $\text{color}[j] \leftarrow R_S + R_L$ ;  $\text{tag}[\text{color}[j]] \leftarrow S$  // new small
      else
         $\text{color}[j] \leftarrow \text{free}_L.\text{POP}()$  // reuse large (forced)

  return  $R_S + 2R_L$ 

```

The idea: when a $B[j] = 0$ interval needs a room and no small room is free, we open a new small room instead of handing it a large room. This saves large rooms for future $B[i] = 1$ intervals. We only fall back to reusing a large room when we have already opened D rooms and cannot open more.

Proof:

(A) **Total rooms = D :** A new room is opened in two cases: (i) both stacks are empty, meaning every existing room is in use, so the current depth $\geq$ total rooms and total stays $\leq D$; or (ii)

$B[j] = 0$ with $\text{free}_S = \emptyset$ and $\text{total} < D$, so $\text{total stays} \leq D$. Since any valid coloring needs $\geq D$ rooms, we get $R_S + R_L = D$.

(B) $R_L = D_1$: Since $\text{cost} = D + R_L$, minimizing cost is equivalent to minimizing R_L .

$R_L \geq D_1$: at a time when D_1 intervals with $B[i] = 1$ overlap, each sits in a distinct large room.

$R_L \leq D_1$: call a room *tainted* the moment it first becomes large (created large or promoted). Only a $B[j] = 1$ interval can taint a room (when $\text{free}_L = \emptyset$). A $B[j] = 0$ interval never taints: it either reuses a free small room, opens a new small room (when $\text{total} < D$), or reuses a room that is already large.

Consider the r -th tainting, caused by some $B[i] = 1$ interval I_i . Since $\text{free}_L = \emptyset$, all $r - 1$ existing large rooms are occupied by intervals that overlap $S[i]$. We claim each of those occupants has $B = 1$. Indeed, the only way a $B[j] = 0$ interval sits in a large room is through the forced-reuse case ($\text{total} = D$ and $\text{free}_S = \emptyset$). But that means *all* D rooms were occupied, so no room was free at all, contradicting $\text{free}_L \neq \emptyset$.

So at time $S[i]$ there are $r - 1$ overlapping $B = 1$ intervals plus I_i itself, giving $d_1(S[i]) \geq r$ and hence $R_L \leq D_1$.

$\therefore R_L = D_1$, $\text{cost} = D + D_1$, matching the lower bound. □

Runtime: Two passes over $2n$ sorted events, each $O(n)$ + sort $O(n \log n) = \boxed{O(n \log n)}$